



Ursinus
COLLEGE

CS375 Spring 2026

Ralph Rostock

Week 12

Software in Production, Documentation,
and The Role of AI in Software Engineering



Ursinus
COLLEGE

Team Standup Meetings



Last Week

Maintenance, Refactoring, and Technical Debt



Software in Production

It shipped. Now what?

- Shipping is not the finish line, it's when the software starts its real life.
- Users do things you didn't anticipate. Infrastructure behaves differently than dev.
- Dependencies change underneath you: libraries, APIs, the OS, your cloud platform.
- The codebase you shipped at v1.0 will be maintained by someone who wasn't there when it was built.

Everything from here is about managing a living system, not a finished artifact.



The 2am page

- Production is down. You are on call. Your phone rings at 2am.
- The first question is never “what went wrong” – it’s “what do I know right now”
- Your logs, your metrics, your alerts are the only window into a system you cannot directly observe.
- Every decision you make in the next 30 minutes is made under uncertainty, with real users affected.

This is the scenario that all of monitoring, alerting, and incident response exists to survive



Logging: your window into a running system

- A log is a timestamped record of something that happened: an event, an error, a state change.
- Good logs answer: what happened, when, to which user, with what inputs
- The failure mode: logging too little (you can't diagnose) or too much (signal lost in noise).
- Structured logs (JSON, key-value pairs) are searchable and machine-readable — plain text logs are not
- Log levels matter: DEBUG during development, INFO for normal events, WARN for recoverable problems, ERROR for failures

Rule of thumb: if you'd want to know it during an incident, log it. If you wouldn't, don't.



Monitoring and alerting

- **Monitoring:** continuously measuring the health of a running system
- **Key metrics to watch:** error rate, response time, resource usage (CPU/memory), and business metrics (active users, transactions)
- **The difference between a dashboard and an alert:** dashboards tell you what's happening, alerts wake you up when you need to act
- **Alert on symptoms, not causes** — 'error rate above 5%' is actionable; 'CPU above 80%' might not be
- **Alert fatigue is real:** too many low-signal alerts and on-call engineers stop responding to them

The goal is confidence: to know the system is healthy without having to constantly check



Incident Response

- An incident is any unplanned interruption to a service or degradation in quality
- The phases: detect → acknowledge → investigate → mitigate → resolve → document
- Mitigation is not the same as resolution — sometimes you roll back first and fix later
- Communication matters as much as technical response: who knows what, and when
- The post-mortem: after the incident, what happened, why, and what changes to prevent recurrence

Blameless post-mortems: the goal is to fix the system, not to find someone to blame



Deployments and Rollbacks

- Every deployment is a risk. The goal of deployment practices is to make that risk manageable
- Feature flags: ship code without activating it, then turn it on for a subset of users
- Canary releases: deploy to a small percentage of servers first, watch for problems, then expand
- Blue-green deployments: keep the old version running until the new one is confirmed healthy
- The ability to roll back quickly is as important as the ability to deploy quickly

A team that can deploy 50 times a day safely is more reliable than one that deploys once a month nervously



Support and the Feedback Loop

- User-facing bugs often surface through support before monitoring catches them
- Good support practice means capturing enough context to reproduce: steps, environment, what was expected, what happened
- Support tickets are a product signal, not just a bug queue — patterns in tickets reveal usability problems, unclear documentation, missing features
- The loop: support ticket → triage → reproduce → fix → verify → close → document if recurring
- Technical teams that ignore support tend to build the wrong things and fix the wrong bugs

A team that can deploy 50 times a day safely is more reliable than one that deploys once a month nervously



Why Documentation Almost Always Fails

- It's written for the author, not the reader — the author knows the system, so they describe it from the inside out
- It's written at the wrong time — after the code is done, under deadline pressure, by someone who just wants to be finished
- It has no clear audience — 'documentation' is one word for at least four different things with four different readers
- It's treated as a one-time artifact — code changes, documentation doesn't, and the gap becomes a trap

This is a structural problem, not a laziness problem. Knowing that, you can design around it.



Orientation vs Reference

- Orientation: what is this, why does it exist, how do the pieces fit together, where do I start
- Reference: specific facts looked up on demand — API parameters, config options, error codes
- Most documentation is written as reference. Most desperate readers need orientation first
- A new developer on your project needs orientation before any reference is useful
- The README is almost the only orientation document most projects have — and it's usually terrible

Could someone with your skills but none of your context understand this project in 20 minutes?



Four types of documentation

- **Tutorials — learning-oriented**
Hand-holds the reader through their first experience. Goal: build confidence.
Ex: getting started guide
- **How-to guides — task-oriented**
Step-by-step for a specific goal. Reader knows what they want, just not how.
Ex: how to deploy, how to reset a password
- **Reference — information-oriented**
Accurate and complete. Consulted, not read.
Ex: API docs, config options, CLI flags
- **Explanation — understanding-oriented**
The 'why.' Context, reasoning, design decisions.
Ex: why we chose this architecture, how the system fits together



User Documentation: The Harder Problem

- Technical documentation describes the system. User documentation describes what the user can accomplish with it
- These require genuinely different skills — and a different kind of forgetting
- You have to forget that you built it. You have to forget what all the terms mean. You have to forget the obvious things
- The practical test: hand your user docs to someone who has never seen the project. Watch where they stop
- Where they stop is where you assumed knowledge you shouldn't have assumed

User documentation is an act of empathy. It asks: what does it feel like to encounter this for the first time?



Documentation that lives with the code

- Documentation that lives outside the codebase — a Wiki, a Google Doc, a separate repo — almost always goes stale
- Documentation that lives in the repo, reviewed with pull requests, gets updated when the code changes
- The README is the entry point. It should pass the '10-minute test': a new developer running the project in 10 minutes
- Architecture Decision Records (ADRs): short documents capturing why a significant choice was made
 - Format: 'We decided to use X instead of Y because Z. The tradeoffs we accepted are...' Even one paragraph counts.
- Code comments: explain why, not what. The code says what it does. Comments say why it does it that way



What AI changes about documentation

- Generating a plausible README is now trivial — AI can produce one in seconds from a codebase
- This lowers the floor: there's no excuse for having no documentation
- But it raises the bar for what documentation is worth reading: generic plausible text is now cheap
- The question is no longer 'do you have documentation' — it's 'does your documentation say anything true and specific that an AI couldn't have made up'
- AI-generated docs tend to be orientation-shaped but reference-thin and explanation-absent

The irreplaceable documentation is the 'why' — the context only you have because you made the decisions



The Why Layer: Intent as Infrastructure

- Code tells you how a problem is solved. It rarely tells you why that problem mattered
- Over time, behavior accumulates without explanation — rules that might be compliance, contracts, or workarounds. Or might not be. We no longer know
- AI-assisted development makes this worse: it optimizes behavior, but cannot preserve intent it was never given
- If you ask a machine to refactor a system whose rationale is undocumented, it will confidently simplify away rules that exist for reasons no one remembers
- The fix isn't more documentation — it's deliberate capture of intent: ADRs, obligation registers, domain models tied to regulatory sources
- *"Writing down why something exists is not a step backward. It is what allows systems to evolve without losing themselves along the way."* — Claus Jepsen, CTO at Unit4

<https://www.linkedin.com/pulse/why-layer-intent-missing-infrastructure-enterprise-software-jepsen-d4n2e/>



The Cost of a Bug: A History

- Windows 95 shipped on physical discs. If there was a bug post-release, patching meant shipping new discs
- Very few people had internet fast enough to download an OS patch. So most bugs, once shipped, stayed shipped
- This meant the economics strongly favored catching bugs before release: massive QA, long testing cycles, cautious release practices
- The cost of a post-release bug was enormous. The cost of pre-release prevention was justified

The shape of the software delivery pipeline shaped every decision about how to build software



The Internet Changed the Calculus

- When the internet became ubiquitous, software updates became cheap to distribute
- Bugs became cheaper — not because code got better, but because you could fix and redeploy
- The ROI of exhaustive pre-release testing shifted: why spend months preventing every bug if you can fix most in hours?
- This enabled continuous delivery: ship often, fix fast, iterate based on real user behavior
- The investment in massive QA teams and long release cycles started to look like overengineering

The change in delivery mechanics changed what it was rational to invest in.



AI Moves the Bottleneck Again

- For decades, engineering has been the most expensive constraint in the software pipeline
- Organizations optimized everything upstream — requirements, design, product decisions — to protect that expensive engineering time
- AI is making the act of coding significantly cheaper and faster — potentially 50-100% throughput improvement
- When the engineering bottleneck loosens, the constraint moves upstream: to product, design, and decision-making
- The pipeline that was built around engineering as the bottleneck will break — and then get redesigned

Releasing a bottleneck doesn't solve your problems. It reveals the next one.



Speed over Risk Mitigation

- When engineering throughput was scarce, the rational move was to minimize risk before it hit engineering
- When engineering throughput is abundant, the rational move shifts toward volume and iteration speed
- Companies will likely ship more features that fail — and accept that trade because volume makes up for individual failures
- Risk mitigation doesn't disappear: catastrophic failures (data breaches, safety issues) still demand it
- But a lot of what was called 'risk mitigation' was actually just 'protecting expensive engineering time'

The question becomes: what kind of risk are you actually managing, and is the cost of managing it still justified?



Software Engineering as a Career

- Writing Code is the skill that's becoming less scarce.
- Judgement , problem decomposition, understanding what needs to be built and why are skills that remain scarce.
- Engineers who can make good product decisions on the fly, without a formal handoff from a PM, become more valuable
- The new role may look less like 'developer who attends product meetings' and more like 'someone who holds product, design, and engineering in one head'

“Punching cards didn't define engineers then. Manually typing code doesn't define you now. Your way of approaching problems is what makes you an engineer.”

Recommended listening: 'AI Moves the Bottleneck' — Developer Tea, Jonathan Cutrell (March 2026)



Assignment: Weekly Standup Reflection

<https://rrostock1.github.io/Ursinus-CS375/Assignments/StandupReflection>

(Individual Assignment)

Due Feb 26 (and every week thereafter, ending tonight).



Assignment: The Overdose

<https://rrostock1.github.io/Ursinus-CS375/Assignments/Overdose>

(Individual Assignment)

Due Apr 23



Assignment: Software Test Plan

<https://rrostock1.github.io/Ursinus-CS375/Project/TestPlan>

(Group Assignment)

Due Apr 23



Participation: Second Self-Assessment

<https://rrostock1.github.io/Ursinus-CS375/Assignments/Participation/SelfAssessment>

(Individual Assignment)

Due May 7



Assignment: Software Documentation

<https://rrostock1.github.io/Ursinus-CS375/Project/DocumentationFinalReport>

(Group Assignment)

Due May 7



Final Presentation: Code and Presentations Due

<https://rrostock1.github.io/Ursinus-CS375/Project/Presentation>

(Group Assignment)

Due (Tuesday) May 12, 7:00 PM



Peer Review: Peer Review Survey Due

<https://rrostock1.github.io/Ursinus-CS375/Assignments/PeerReview>

(Individual Assignment)
Due May 12 (Tuesday)